

Assignment #1 - Client-side programming with JavaScript

Due: Thursday, October 8th at 11:59 PM

Overview

In this assignment, you will develop a web site that allows a user to browse through various pets that are available for adoption from several animal shelters. The user will be able to select shelter, select a pet, view the pet's details and then apply to adopt the pet.

Everything you implement for this assignment will involve client-side programming within the browser. You must implement everything using plain **JavaScript** and you **may not** use external libraries/tools such as **jQuery** and **Bootstrap**. Before starting your design for the assignment, you are encouraged to read through the entire specification. It is possible that later problems will inform your design decisions and, by preparing in advance, you can avoid having to significantly refactor your code as you progress through the assignment.

This assignment has been broken up into three parts that correspond to the completed tutorials. If you want to do well in this course, follow the directions given on each assignment and time things carefully with the tutorials as indicated here:

- **PART A** – Start this section as soon as you have completed **Tutorial 1**.
- **PART B** – Start this section as soon as you have completed **Tutorial 2**.
- **PART C** – Start this section as soon as you have completed **Tutorials 3 & 4**.

Assignment Resources

- File: [shelterData.js](#) – contains a **JavaScript** object representing all shelters with their pets and information
- File: [aboutData.js](#) – contains a **JavaScript** object containing information needed for the [about.html](#) page
- Folder (from [images.zip](#)): **images** – a set of images referred to in the pet data. The images supplied in this assignment were all created using **openart.ai**. You may use these images or replace them with your own.

To begin the assignment, download above resources from Brightspace. The [shelterData.js](#) file contains an array (i.e., **shelters**) of 5 animal shelter objects containing the data for each shelter. Each shelter contains five properties:

- **id** – a unique ID for the shelter
- **name** – the name of the shelter
- **city** – the city where the shelter is located
- **processingFee** – a standard base fee that is applied to all adoptions
- **pets** - an object array containing all of the pets up for adoption at that shelter

Each pet in the **pets** array contains the following information: **id**, **name**, **type**, **breed**, **age**, **health**, **traits**, **image**, **views**, **adoptionFee** and **daysAtShelter**.

The [aboutData.js](#) file contains a **mission** statement paragraph, a brief **teamDescription** sentence and an array (i.e., **team**) of team members, where each team member has a **name**, **role**, **experience** and **qualifications**.

You may use the data as it is provided, or you may re-format the data in any way you desire (or add to it), but you must keep at least the specified attributes.

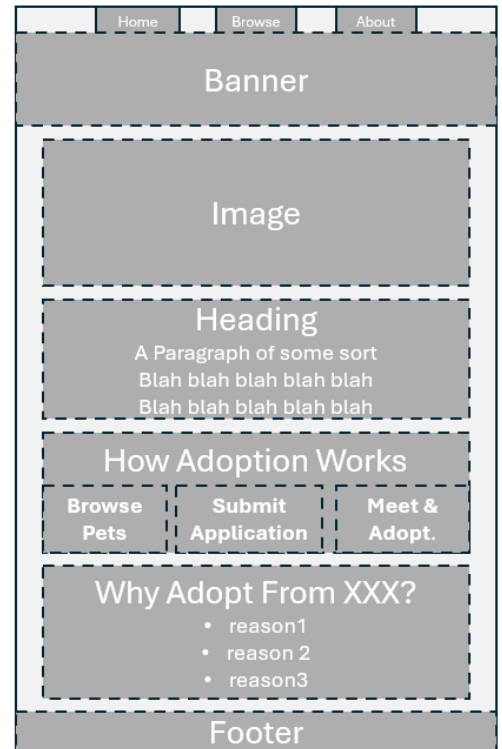
- ***IMPORTANT*** – For all A.I.-generated images, please indicate in a README file what software/site you used to generate them. If you are making use of images found online, please acknowledge in your readme where you found them.

PART A – HTML (Complete as soon as you finish Tutorial 1)

The [index.html](#) Page

The website's landing page (i.e., the main starting page for the site) must be an **HTML** page called [index.html](#) and it **MUST** have the following (see drawing here for one possible way of laying things out on the page, but you can arrange things however you would like):

- A navigation bar with three options: **Home**, **Browse Pets** and **About Us**. Selecting **Home** on the navigation bar should revert to this [index.html](#) page. Selecting **Browse Pets** must go to [browse.html](#) page (described soon). Selecting **About Us** must go to [about.html](#) page (described soon).
- A banner **image** at the top that shows the **name** of the company (the name is shown as **XXXX** below), which should have something to do with animal rescue or adoption ... pick something interesting. Also, show some kind of short one-line company slogan under the company name that is in smaller font (or as part of the banner image). Perhaps you have access to A.I. image-generating software. I use the **openart.ai** website, but you may have your favourite. You can use that to make a banner with your company logo and slogan and have some kind of image related to animal adoption. Your banner image should be about 3 to 4 times wider than its height. If you are unable to find a place to create/generate an image, you may simply use a text-based header with the company name and slogan, but make sure to style it nicely.
- A **title** displayed on the browser tab that corresponds to the name of the company (i.e., **XXXX**).
- Some kind of welcoming catch-phrase **heading** and a **paragraph** that explains the benefits of adopting a pet (use ChatGPT for ideas to generate the text). You should also include a nice visual eye-catching image depicting the pleasant side of adopting.
- A **How Adoption Works** section on the page with three steps: **Browse Pets**, **Submit Application** and **Meet & Adopt**. A **Why Adopt From XXXX?** section that lists a few reasons why someone should adopt from this company (you may use ChatGPT for ideas to generate some text and then adjust accordingly).
- A page **footer** that displays as © 2026 **XXXX** | **SSSS**, where **XXXX** is your company name and **SSSS** is your company slogan (i.e., the information in the header/banner).
- You may add other elements on the page if you would like.



The [about.html](#) Page

The [about.html](#) page must meet the requirements listed below.

- The page should have a **title** displayed on the browser tab that corresponds to the name of the company followed by something indicating that we are on the **About Us** page (e.g., "**XX** – About"). Here, **XX** may be the company name that you have chosen earlier, part of that name or an abbreviation of it if it is too long).
- The page should have the same exact **navigation** bar as the landing page.
- The page should have the exact same **banner** image (or text header) and **footer** as the landing page.
- The page should have an **Our Mission** heading and then display the mission paragraph from [aboutData.js](#). For now, hard-code (i.e., copy/paste) this onto your page.
- The page should have an image depicting the company (e.g., generate an A.I. image of an animal shelter with people working in it, or find something online and be sure to reference it in your readme).
- The page should have an **Our Team** heading and then display the **teamDescription** statement followed by a nicely organized layout of the **team** members. For now, hard-code (i.e., copy/paste) this information onto your page with the text from [aboutData.js](#).

The [browse.html](#) Page

The [browse.html](#) page must meet the requirements listed below:

- When loaded, the page should have a **title** displayed on the browser tab that corresponds to the name of the company followed by something indicating that we are on the **Browse Pets** page (e.g., "**XX** – Browse"). Here, **XX** may be the company name that you have chosen earlier, part of that name or an abbreviation of it if it is too long).
- The page should have the same exact **navigation** bar as the landing page.
- The page should have the exact same **banner** image (or text header) and **footer** as the landing page.
- The page should have a **Select a Shelter** heading followed by a drop-down list that is populated from the shelters in [shelterData.js](#). For now, you can hard-code the options in the list from the shelter names in [shelterData.js](#). For now, assume that the first shelter is selected upon page load.
- The selected **shelter's name** must be visible ... you can simply have it visible in the drop-down list.
- A list of exactly **6** buttons should appear to represent the type of pets to be shown. The buttons should be labelled **All**, **Cats**, **Dogs**, **Birds**, **Reptiles** and **Other**. The buttons should behave like a group of radio buttons (in that only one can be on at a time). However, assume that the **All** button is selected for now.
- All **pets** that belong to the currently-selected **type** button should be **displayed** as a `<section>` on the main page with some kind of *grid* layout that wraps around as the page is resized. Each pet should be displayed in a `<div>` within this `<section>` showing each pet's **image**, **name** and a **View Details** button. For now, you should hard-code all the pet data from the first shelter's data in [shelterData.js](#) (i.e., "Happy Tails Rescue").
- When a **View Details** button is pressed, a [pet.html](#) page should be loaded (see next section).

The [pet.html](#) Page

The [pet.html](#) page MUST meet the following requirements:

- The page should have a **title** in the browser tab showing "**XX** - Pet Details", where **XX** is either the company name that you have chosen earlier (or part of the name or an abbreviation if it is too long).
- The page should have the same exact **navigation** bar as the landing page, except that the **Browse Pets** will show as selected when this page loads.
- The page should have the exact same **banner** image (or text header) and **footer** as the landing page.
- For now, assume that the first pet from the first shelter has been selected (i.e., see "Milo" in [shelterData.js](#)).
- You should display the pet's image. The **name** of the pet should also be shown as a large header text. Then the **age**, **breed** and **type** must be shown (e.g., "2 year old Siamese (Cat)"). The **health**, and the list of **traits** should be shown separated in some way. The number of **daysAtShelter** should also be shown.
- The **adoptionFee**, shelter's **processingFee** and their **total (before taxes)** (i.e., the sum of the two fees) should also be shown, followed by a **button** labelled "**Apply to Adopt this Pet**".
- Some kind of "**← Back to Browsing**" link should be placed strategically on the page such that when pressed, it requests [browse.html](#).
- The **Apply to Adopt this Pet** button should, when pressed, cause the [apply.html](#) page to be loaded (see next section).

The [apply.html](#) Page

The [apply.html](#) page MUST meet the following requirements:

- The page should have a **title** in the browser tab showing "**XX** - Application", where **XX** is either the company name that you have chosen earlier (or part of the name or an abbreviation if it is too long).
- The page should have the same exact **navigation** bar as the landing page, except that the **Browse Pets** will show as selected when this page loads. The same kind of hover effect must be there.
- The page should have the exact same **banner** image (or text header) and **footer** as the landing page.
- The page should have a single `<form>` element that contains the following:
 - A `<div>` for the pet's information that will have some nicely formatted text that indicates the **name**, **type** and **breed** of the pet being requested as well as a breakdown of the fees, showing the **adoptionFee**, the **processingFee** and the total fees (before taxes). For now, fill this in with the first pet (i.e., "Milo") from the first shelter.
 - A section called **Your Information** that will have fields for the user's **full name**, **email address**, **phone number**, **home address** and **number of residents in the home**. It should be made clear to the user as to what goes into each field.
 - A section called **Current Pets in Your Home** that will have a text area where the user can enter text-based information. Some default text/prompt should be there to indicate an example of what goes in here (e.g., List other pets that you have (species, age, health)).
 - A section called **Pet Experience** that will ask the user if they have experience with this kind of pet and allowing them to select **None**, **Some** or **Extensive** from a set of dropdown options. This section will also ask the user where this pet will primarily live, allowing them to select **Indoor**, **Outdoor** or **Both** from a set of dropdown options.

- A section called **Support Us** that will allow the user to select an optional donation of \$0, \$10, \$25, or \$50 from a set of dropdown options.
- A **checkbox** with the statement that the user agrees to provide proper care, food, shelter and medical attention for the pet.
- A **checkbox** with the statement that the user understands that the application does not guarantee adoption.
- A **Submit Application** button.

PART B – CSS (Complete as soon as you finish Tutorial 2)

Do not put any inline-styling on any of your pages. Instead, create external cascading stylesheets (i.e., **.css** files) as necessary. You may make one that contains general styling for all pages and others that are specific for each page. You can ask ChatGPT to produce an initial styling template based on a theme according to your code and your prompt description. Make sure to carefully examine the produced file so that you fully understand it. You will need to tweak it a little to get it to work and to meet the assignment requirements. Overall, you must ensure that the **look & feel** is consistent across ALL pages.

The navigation links

Your styling must meet the following requirements:

- The navigation buttons should have some kind of visual **hover** effect for the links that are not selected.
- Selecting **Home** on the navigation bar will cause it to show as "selected" in some way. This should be selected when the page loads.
- Selecting **About Us** on the navigation bar will cause it to show as "selected" in some way.
- Selecting **Browse Pets** on the navigation bar will cause it to show as "selected" in some way.

The index.html Page

This page is quite flexible in styling, but you should ensure that things are grouped according to the diagram in PART A. It is a good idea to make things centered as needed and to group things into card-like-boxes with reasonable margins.

The about.html Page

The **about.html** page must meet the requirements listed below:

- The **About Us** will show as selected when this page loads.
- The page should have the exact same **banner** image (or text header) and **footer** as the landing page.
- The page should be able to adjust nicely regardless of the number of team members in the data (e.g., assume at least 1 and less than 10).

[The browse.html Page](#)

The [browse.html](#) page must meet the requirements listed below:

Whenever a shelter has been selected (or the user changes the shelter from the list), the following additional requirements must be met. Keep in mind that these requirements should be handled by writing client-side **JavaScript** code and **CSS** code ... which must all be written in external files (i.e., not using inline **JavaScript** nor inline **CSS**).

- The pet-type buttons (i.e., **All**, **Cats**, **Dogs**, **Birds**, **Reptiles** and **Other**) should behave like a group of radio buttons (in that only one can be on at a time) and have the following features:
 - The **All** button should appear selected button and to indicate selection, it must have a different color than the other buttons.
 - If the shelter does NOT have any pets of a **type** that corresponds to one of the 6 buttons, then it must be **disabled** in some way so that the user cannot select it. It must appear visibly disabled in some way. For now, this should be the **Reptiles** and **Other** buttons, since we are temporarily assuming that the first shelter as been selected.
 - If there is at least one pet whose **type** corresponds to a button's type, then this button should be enabled, showing with a different color than the currently **selected** button. Also, all enabled buttons must have some kind of hover effect, unless it is the currently-selected button, which should have no hover effect applied (also, no hover effect should occur over disabled buttons). For now, this will be for buttons **All**, **Cats**, **Dogs** and **Birds**, since we are assuming shelter 1 is selected.
- Make sure that your *grid* of pets wraps around as the page is resized.
- Adjust so that each pet image is displayed with a size of no more than 120 x 120 pixels.
- Each pet `<div>` should have some kind of **hover** effect so that the user gets a visual cue as to which pet the cursor is on (there are examples in the course notes that do similar things).
- The **View Details** button should also have a visible **hover** effect.

[The pet.html Page](#)

The [pet.html](#) page MUST meet the following requirements:

- The pet's image should be displayed at least 3 times larger than it was displayed on the [browse.html](#) page.
- The **Apply to Adopt this Pet** button should also have a visible **hover** effect and when pressed.

[The apply.html Page](#)

The [apply.html](#) page MUST meet the following requirements:

- The page should be made to look like a single form to be filled out (similar to a paper-based form).
- A **Submit Application** button should be disabled to start. When enabled, the button should have some kind of hover effect consistent with the look & feel of the site.

PART C – JavaScript & DOM (Complete as soon as you finish Tutorials 3 & 4)

The [about.html](#) Page

The [about.html](#) page must meet the requirements listed below when the page loads. To make this happen, you MUST create an [about.js](#) JavaScript file that dynamically populates the HTML in [about.html](#) based on data from [aboutData.js](#). So, you must modify your [about.html](#) file to use only placeholders for this information (i.e., remove the hard-coded text for the `mission` paragraph, `teamDescription` and `team members`). You may NOT use inline JavaScript.

- Adjust the [about.html](#) so that it loads the external [about.js](#) script.
- Write code in [about.js](#) so that the page dynamically loads the `mission` paragraph from [aboutData.js](#).
- Write code in [about.js](#) so that the page dynamically loads the `teamDescription` from [aboutData.js](#).
- Write code in [about.js](#) so that the page dynamically loads the `team members` from [aboutData.js](#).
- Remember that the page should be able to adjust nicely regardless of the number of team members in the data (e.g., assume at least 1 and less than 10).

The [browse.html](#) Page

The [browse.html](#) page must meet the requirements listed below when the page loads. To make this happen, you MUST create an [browse.js](#) JavaScript file that dynamically populates the HTML in [browse.html](#) based on data from [shelterData.js](#). So, your [browse.html](#) file must have only placeholders for this information. You may NOT use inline JavaScript.

- The drop-down list must be populated from the shelters in [shelterData.js](#). That is, it must be loaded dynamically using JavaScript (i.e., you must remove your hard-coded name selections from the drop-down list). No shelter should be selected upon page load. It should be clear that the user needs to select a shelter to see the pets. When no shelter is selected in the drop-down list, nothing else is to be shown on the page (i.e., no pet types nor pets).
- When a shelter is selected, the 6 pet-type buttons should always be shown, regardless of the shelter. Ensure that the buttons behave like a group of radio buttons (in that only one can be on at a time) and have the following features:
 - The **All** button should be **selected** by default whenever the user changes the shelter from the dropdown list.
 - If the shelter does NOT have any pets of a **type** that corresponds to one of the 6 pet-type buttons, then it must be **disabled** in some way so that the user cannot select it. It must appear visibly disabled in some way.
 - If there is at least one pet whose **type** corresponds to a button's type, then this button should be enabled, showing with a different color than the currently **selected** button.
- Make sure that all **pets** that belong to the currently-selected **type** button are **displayed**.
- Make sure that whenever a **View Details** button is pressed, the [pet.html](#) page is loaded. Both the **pet ID** and the **shelter ID** must be **sent along with the URL as query parameters** when the [pet.html](#) page is requested.

The [pet.html](#) Page

The [pet.html](#) page should meet the following requirements:

- We will no longer assume that "Milo" has been chosen, so you will need to replace the hard-coded information for "Milo" with dynamic pet information based on the chosen pet (i.e., based on the incoming **pet ID**).
- For the **pet** that was chosen (i.e., **pet ID**), you should make sure to use the chosen pet's **image**, **name**, **age**, **breed**, **type**, **health**, list of **traits** and **daysAtShelter**.
- You will also need to adjust the page to show the chosen pet's **adoptionFee** and the selected shelter's **processingFee** (i.e., based on the incoming **shelter ID** parameter).
- When the "**← Back to Browsing**" link is selected, the **shelter ID** should be sent along with the URL (i.e., to [browse.html](#)) as a query parameter.
 - You will need to adjust your [browse.js](#) code so that it looks for this **shelter ID** query parameter and ensures that this shelter is selected (and its type buttons and pets displayed) when returning to this page from the [pet.html](#) page. Note that whenever **Browse Pets** is selected in the navigation menu from ANY page, it always reverts back to no selected shelter (and hence no type buttons nor pets will be displayed).
- The pet's **ID**, **name**, **type**, **breed** and **adoptionFee** as well as the shelter's **processingFee** must be sent along with the URL as query parameters when the [apply.html](#) page is requested (i.e., when the **Apply to Adopt this Pet** button is pressed).

The [apply.html](#) Page

The [apply.html](#) page MUST meet the following requirements:

- Instead of hard-coding Milo's information, the form's placeholder `<div>` should be filled in dynamically from an external [apply.js](#) JavaScript file that accesses the URL query parameters sent from the [pet.js](#) when the **Apply to Adopt this Pet** button is pressed.
- The **Submit Application** button should be disabled unless both checkboxes have been checked. If they have both been checked, the button should be enabled physically and visually.
 - When the button is pressed, your [apply.js](#) code must dynamically change the page by removing the form and showing a message indicating that the application was submitted successfully and a polite **Thank You** statement letting the user know that the application will be reviewed and that they will be contacted soon.
 - There should also be a **Return to Home** button appearing after the message that will have the same look & feel as the site and it should re-direct the user to the landing page.

Overall Site Quality (10%)

These marks will be allocated for the overall quality of your site's implementation. This will include factors such as the visual appeal of your pages, as well as the responsiveness of your pages to user actions and changes.

For example:

- How do your pages react to the changing of the browser dimensions?
- Is information easy to find on your pages?
- Is everything on each page formatted in a nice way?

So, make sure that your pages have a nice look & feel (i.e., color theme, margins, font-sizes etc.). To get ideas, look at the webpages and stylesheets used in our examples from the course notes, but come up with your own style. If you struggle to make visually appealing pages, you may use ChatGPT for some help styling.

You are also free to add additional components or functionality to your page. Include a description of any additions/changes you made within your [README.txt](#) file. However, make sure that you at least have the required elements and functionality.

Code Quality and Documentation (10%)

Your code should be well-written and easy to understand. This includes:

- Providing clear documentation explaining the purpose and function of pieces of your code.
- Use of good variable/function names that make your code easier to read.
- Avoiding unnecessary computation
- Ensuring your code runs smoothly throughout operation.

You MUST also include a [README.txt](#) file that explains...

- acknowledgements/references for how you obtained your images and/or text. If you made your own images and wrote all your own text, then indicate this as well.
- additional design decisions that you made beyond what was required,
- any additional instructions that may be helpful to the TA.

Submission Checklist

- ☐ Submit a single zip file called [StudentID-COMP2406-A1.zip](#). (Where [StudentID](#) is replaced by your Carleton ID number).
- ☐ **CHECK:** Your zip file should contain all the resources required for your assignment to run (including images, stylesheets and scripts). The TA should be able to open the [index.html](#) page and view your site without any additional setup. You MUST NOT require any server to be running.
- ☐ **SUBMIT:** Upload your [StudentID-COMP2406-A1.zip](#) file to the **Assignment1** submission on Brightspace.
- ☐ **TEST:** Make sure you download the zip after submitting and verify the file contents. If your zip file is missing files or corrupt, you will lose marks.